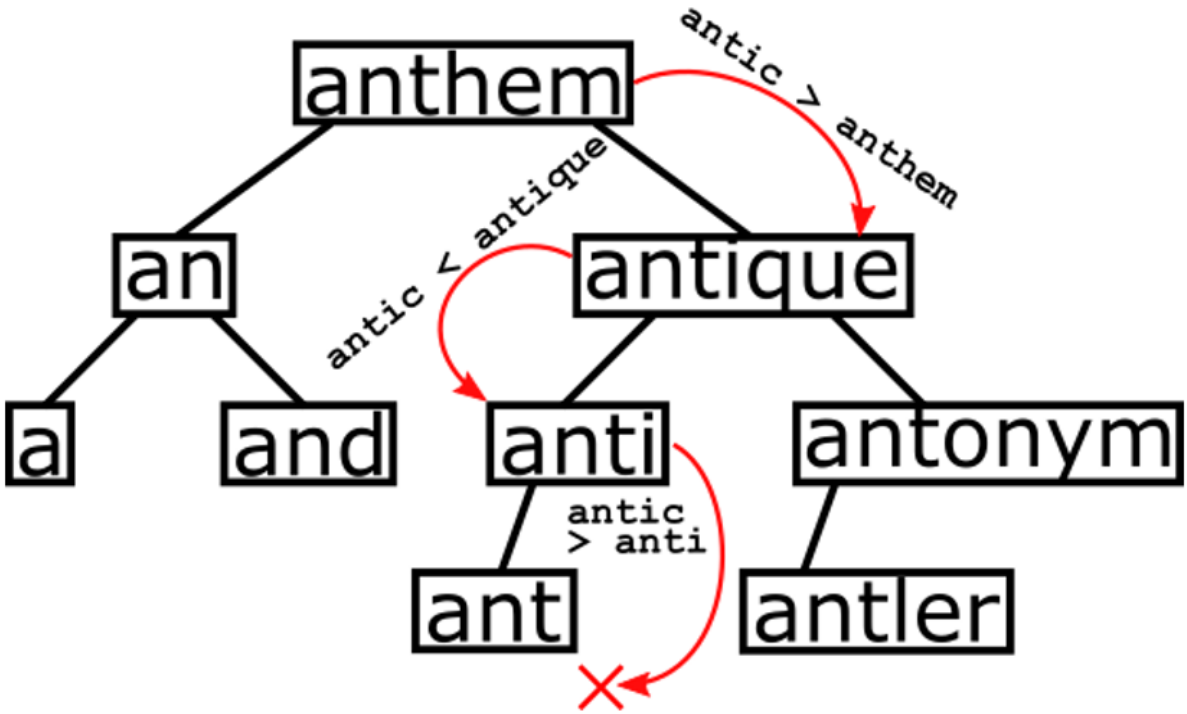


Figure 6.1 A BST storing (part of) a dictionary, and the steps to search the word "`antic`": in this example, it's a miss, as the word was not stored in the tree shown.

So far, however, when we analyzed trees, we have assumed their keys either were integers or that they could be checked in constant time, and require a constant space.

For generic strings this assumption is not realistic anymore: each node will need to store a string of unbounded length, and so the total memory needed to store the tree will be the sum, for each node, of all the keys' lengths:

$$E\left[S\left(n\right)\right] = E\left[\sum_{i=0}^{n-1}\left|k_i\right|\right] = \sum_{i=0}^{n-1} E\left[k_i\right] \approx \sum_{i=0}^{n-1} L = n \cdot L$$